

Performance Profiling and Benchmarking Report

Project: GPU-accelerated Pencil Beam dose reconstruction (IMPT, Fermi-Eyges analytical model) **Data sources:** `results/benchmark_full.json`, `results/gpu_microbench.json`, `results/benchmark_log.txt` **One-line conclusion:** On the full-size clinical case of 128^3 / 67,240 pencil beams, the per-layer physically exact GPU path reaches **7.41 ms** — **155,544×** faster than the serial CPU baseline and 13.5× faster than the 100 ms target; of that, the pure algorithmic contribution (separable convolution) accounts for 1,242×, while hardware and precision (BLAS / multithreading / GPU FP32) account for roughly 125×. Every number comes from measurements on this machine; anything that is an extrapolation or an analytic derivation is explicitly flagged as such in the text.

1. Test Platform and Methodology

1.1 Hardware and Software Platform

Item	Measured / recorded value
GPU	NVIDIA GeForce RTX 5070 Ti Laptop GPU, <code>sm_120</code> (compute capability 12.0), 46 SM, 11.9 GB VRAM
GPU driver / CUDA	Driver 581.42, CUDA 12.8 runtime (<code>torch.version.cuda</code>)
CPU	AMD Ryzen 9 9955HX, 16 cores / 32 threads (PyTorch reports <code>torch.get_num_threads() = 16</code>)
Memory	31.2 GB
Framework	PyTorch 2.11.0+cu128 ; NumPy (float64 reference path)
Custom-kernel backend	Triton 3.8.0 (<code>triton-windows</code>)

How the "custom kernels" are implemented (important limitation): this machine has **no CUDA Toolkit installed (no `nvcc`, no MSVC)**, so `.cu` sources cannot be compiled. The hand-written kernels therefore follow the chain **Triton JIT** → **PTX** → `ptxas` → **sm_120 CUBIN**, and the product is a native GPU kernel equivalent to CUDA C++ (not a library call); the CUDA C++ version in `cuda/pencil_beam.cu` is retained as a code deliverable but was **not compiled and is not counted in this round of measurements**. For the same missing-toolchain reason, this round has **no hardware counter (Nsight Compute) data**: the bottleneck attribution in §5 rests on a roofline model plus microbenchmarks and scaling measurements, so it is inference rather than counter evidence.

1.2 Timing Protocol (implemented strictly as in `pbdose/benchmark.py`)

- GPU timing** uses `torch.cuda.Event(enable_timing=True)` : each measurement creates a pair of events, `s.record() → fn() → e.record()` , followed by `torch.cuda.synchronize()` and a read of `s.elapsed_time(e)` — this avoids host-device synchronisation error and captures the true execution time of asynchronous kernels.
- CPU timing** uses `time.perf_counter()` .

- **Warm-up:** before every measurement the code runs a number of idle iterations to trigger Triton JIT compilation, cuBLAS algorithm selection, memory-pool warm-up and GPU clock boost: 5 iterations for the GPU `bmm` family, 3 for the local windowed gather, 2 for Triton kernels, 2 for PyTorch CPU, and 1 for the NumPy reference.
- **Repeat count:** `python scripts/run_benchmark.py --repeat 15 --tag full`, i.e. **N = 15 repetitions, reporting the median**, with the **minimum** (min, representing best-case performance) recorded alongside it.
- **Run-to-run variance (must be disclosed):** across three complete benchmark runs (`--repeat 15 --tag full`) the median of the headline row (PyTorch bmm per-layer exact FP32) was **6.91 / 6.70 / 7.41 ms** respectively, a spread of roughly **±10%**. That magnitude is consistent with thermal drift and power-limit drift on a laptop GPU, not with any code change — the operators and operation counts of the headline row were completely unchanged between the three runs. Therefore: (a) the **7.41 ms** reported here is the **median of N = 15 repetitions within a single run**, and `min_ms` for the same row is **6.78 ms** (best case); (b) **cross-run differences within ~10% are not significant** and must not be used to claim an optimisation or a regression; (c) every comparison in this report (speed-up, accuracy, roofline) is taken from **the same run** (`benchmark_full.json`), so that the ratios are internally consistent.
- **Exceptions (must be stated):** the NumPy float64 separable reference solution takes 6.8 s per evaluation, so it was actually run with `warmup=1, repeat=3`; the scaling measurement of the Triton naive gather used `warmup=1, repeat=3` (per M point); the naive CPU baseline and the "full-M extrapolation" row were not repeated — they are **extrapolated values** (see §4.1, §4.9).
- **Accuracy comparison:** every GPU/Triton path is cast to float64 before being compared with the CPU float64 reference solution, using the global relative error (divided by the peak of the reference field). The reference field peak dose is **6.352707×10^2** (the fluence-normalised value; the older peak-normalised version was 2.279040×10^1 , and the definition of relative error is unchanged — it is always divided by the reference field peak).

2. Problem Definition

The physical model is a laterally separable pencil beam superposition:

$$D(z,y,x) = \sum_l \sum_{ij} w[l,i,j] \cdot IDD_l(z) \cdot Gx_l(z, x, x_i) \cdot Gy_l(z, y, y_j), \quad Gx_l(z,x,x_i) = 1/(\sqrt{(2\pi) \cdot \sigma_l(z)}) \cdot \exp[-(x-x_i)^2/(2\sigma_l(z)^2)]$$

Key physical constraint: $\sigma_l(z)$ depends on the **local energy** of that energy layer at depth z , $E_l(z) = E(R_l - z)$, so **different energy layers have different σ at the same depth and the lateral kernels cannot be merged across layers** — this is why "per-layer exact" has to perform 40 convolutions, and why the single- σ approximation of §4.6 is off by as much as 20.3%.

Parameter	Value
Voxel grid	128×128×128, 2.0 mm voxels → 2,097,152 voxels (2.10 M), 25.6×25.6×25.6 cm
Lateral spot lattice	41×41, spacing 5.0 mm → 1,681 spots/layer
Energy layers	40
Total pencil beams	M = 67,240 (1,681 × 40)
Energy range	59.1 – 198.0 MeV
CSDA range span	3.00 – 25.50 cm
Naive complexity	$M \cdot N_{\text{vox}} = 1.410 \times 10^{11}$ multiply-adds
Separable complexity	1.135×10^8 multiply-adds (theoretical reduction 1,242.1×)
IDD model	CSDA energy-conserving binning + range straggling $\sigma_R = 0.012 \cdot R^{0.935} + 0.33\%$ energy spread; fluence attenuation length 150 cm; incident beam $\sigma_{x0} = 3.0$ mm; lateral weight envelope Gaussian $\sigma = 2.8$ mm

IDD normalisation convention (corrected this round): the engine's IDD is now **fluence-normalised** — `model.gaussian_impt_field / model.flat_impt_field` obtain their curves through `physics.idd_pristine(..., normalize=False)`, so each layer's IDD carries the **physical unit [MeV/cm]** (**depth dose per unit fluence**) and its peak varies with energy: **43.8 MeV/cm** for the 59 MeV layer versus **19.4 MeV/cm** for the 198 MeV layer (a factor of 2.3; the former is the true peak on a fine depth grid and reads 39.3 MeV/cm when sampled on the 2 mm depth grid). This is a prerequisite for physical correctness: IMPT spot weights are **fluence** (MU / proton count), not peak dose — forcing every layer to a peak of 1 would artificially suppress the relative weight of the low-energy layers by more than a factor of two (see issue 11 in `docs/04_research_log.md`).

Note: the repository carries two op-count conventions — `engines.operation_counts` (counting (2k+1) taps along rows/columns) and `benchmark.roofline_estimate` (counting K^2 taps per voxel) — whose estimates for the windowed gather differ by about an order of magnitude. §5 uses the roofline convention throughout, while §2 and §4 use the `operation_counts` convention.

3. Speed-up Chain Summary

#	Implementation	Device	Precision	Median time [ms]	Min time [ms]	Speed-up vs naive CPU	Max relative error
1	Naive per-beam CPU (extrapolated)	CPU Ryzen 9 9955HX	float64	1,152,984.18	1,152,984.18	1.0×	— (baseline, not compared)
2	NumPy separable (per-layer exact)	CPU single-thread	float64	6,708.46	6,673.72	171.9×	— (reference ground truth)
3	PyTorch bmm (per-layer exact)	CPU 16 threads	float64	395.49	381.83	2,915.3×	5.37×10^{-16}
4	PyTorch bmm per-layer exact (FP32)	RTX 5070 Ti	float32	7.41	6.78	155,543.7×	1.51×10^{-7}
5	PyTorch bmm per-layer exact (TF32 tensor core)	RTX 5070 Ti	tf32	7.35	6.81	156,949.6×	1.51×10^{-7}
6	PyTorch bmm single-sigma approximation (FP32)	RTX 5070 Ti	float32	5.04	4.55	228,894.6×	2.03×10^{-1}
7	PyTorch local windowed gather (FP32)	RTX 5070 Ti	float32	1,003.09	731.63	1,149.4×	2.62×10^{-7}
8	Triton windowed gather + LUT (per-layer exact)	RTX 5070 Ti	float32	82.90	80.25	13,907.7×	1.03×10^{-6}
9	Triton separable convolution (fused Gaussian GEMM)	RTX 5070 Ti	float32	19.59	17.39	58,858.5×	$1.05 \times 10^{-6} +$
10	Triton naive gather (full M, extrapolated)	RTX 5070 Ti	float32	654.19	654.19	1,762.4×	— (baseline, not compared)

† The error of row 9 is implementation fidelity relative to the **single-sigma approximation** reference solution, not relative to the physically exact float64 ground truth.

Note that rows 7–10 are **not on the speed-up ladder**: the local windowed gather and the Triton naive gather are both two orders of magnitude slower than row 4's best exact path; their purpose is to validate the algorithm family and to supply bottleneck evidence (§4.7, §4.9).

4. Step-by-Step Analysis

4.1 Naive per-beam CPU: 1,152,984 ms (extrapolated) — the algorithmic baseline

Implemented as `engines.naive_numpy_per_beam` (NumPy full-grid accumulation beam by beam, not a pure-Python triple loop; the latter is used only for small-scale correctness checks). The full size cannot be run directly, so the measurement used **200 beams × 262,144 voxels = 0.429 s**, extrapolated linearly in $O(M \cdot N_{\text{vox}})$ to 67,240 beams × 2,097,152 voxels. Backing out the effective throughput gives only **0.245**

GFLOP/s, with an effective bandwidth of only **2.94 GB/s** (0.53% of the DRAM microbenchmark): this formulation does not even qualify as "memory-bound" — the algorithm has to be replaced first.

4.2 NumPy separable: 6,708.46 ms, 171.9× — only 13.8% of the pure algorithmic gain was realised

Once the two lateral contractions are written as matrix multiplications, the multiply-add count falls from **1.410×10^{11} to 1.135×10^8 (1,242.1×)**. Yet the measured speed-up is only **171.9×** — just **13.8%** of the theoretical algorithmic gain (a factor of $1242.1/171.9 \approx 7.2\times$ is eaten by implementation inefficiency). The reason is clear: contractions such as `np.einsum("zkji,zkvj->zkiv")` **do not call BLAS GEMM** in NumPy; they take an element-wise, memory-bound path (measured effective throughput **0.0338 GFLOP/s**), and the write-read-reread of the intermediate `T` (26.8 M elements) consumes far more bandwidth than the roofline minimum-traffic model allows. This step establishes that **the value ceiling of the algorithmic change alone is 1,242×**.

4.3 PyTorch bmm CPU: 395.49 ms, 2,915.3× (17.0× over the previous step) — BLAS + multithreading

The algorithm is completely unchanged; the same set of contractions is simply handed to `torch.bmm` (batch = $L \cdot n_z = 5,120$) and executed by MKL/OpenBLAS on **16 threads**. The entire gain comes from BLAS blocking/microkernels, multithreaded parallelism and the way batched GEMM amortises scheduling overhead. Precision is still float64, with a maximum relative error of 5.37×10^{-16} (reference-implementation-level rounding); effective throughput is 0.574 GFLOP/s — even with BLAS, the CPU remains far from its theoretical peak.

4.4 PyTorch bmm GPU FP32 per-layer exact: 7.41 ms, 155,544× — the headline result

The same algorithm moved to the GPU (small matrix multiplies with batch = 5,120, float32) is **53.4×** faster than the multithreaded CPU version of the same algorithm, **155,544×** faster than the naive CPU baseline, and **13.5× faster than the 100 ms target**. Maximum relative error **1.51×10^{-7}** , RMS 1.33×10^{-9} — that is, the "per-layer physically exact" path (with no cross-layer σ merging whatsoever). Back-computing the effective bandwidth from the roofline minimum traffic of 1.153 GB gives **170 GB/s (at min 6.78 ms) / 156 GB/s (at median 7.41 ms)**, i.e. only 28–31% of DRAM bandwidth is used. It is the **only implementation in this project that satisfies both hard constraints at once** — "< 100 ms" and "physically exact".

4.5 TF32: 7.35 ms — no tensor-core speed-up this round, reported as measured

With `allow_tf32` enabled the median time is 7.35 ms, only **0.9% (speedup_vs_prev = 1.0090)** away from 7.41 ms (a difference within the run-to-run variance described in §1.2), and **max_rel_err and rms_rel_err are bit-for-bit identical to the FP32 row (1.51×10^{-7} / 1.33×10^{-9})**. The fact that the error does not change at all is itself the evidence: had TF32 tensor cores really been used, the 10-bit mantissa would have produced relative errors on the order of 10^{-3} .

Two possible causes; this report makes no further assertion: 1. **The shapes are too small**: each layer's bmm is `(41×41)@(41×128)` and `(128×41)@(41×128)`, with a contraction dimension K of only 41; the microbenchmarks show that a small-batch bmm of $41 \times 41 \times 128$ tops out at only **4.11 TFLOP/s** at batch 5120 and only **0.63 TFLOP/s** at batch 128, so the relative advantage of tensor cores was always going to be

limited on such extremely small-K shapes; 2. **API migration**: torch 2.11 has moved `torch.backends.cuda.matmul.allow_tf32` to the new `fp32_precision` interface, so the legacy switch may simply be ignored on this build (this machine cannot confirm either way with nvcc/Nsight).

This report therefore claims no tensor-core speed-up whatsoever, and the TF32 and FP32 rows are treated as equivalent in performance.

4.6 Single-sigma approximation: 5.04 ms (228,895×), but a maximum relative error of 0.203 — clinically unusable

Merging the 40 layers' $\sigma_l(z)$ into a single $\sigma(z)$ weighted by layer weight needs only one expansion instead of 40 layer convolutions, bringing the time down to 5.04 ms (only **1.47×** relative to 7.41 ms). The price is a **maximum relative error of 20.3%** and an RMS relative error of 3.78×10^{-3} . The physical reason is that σ 's range dependence on energy is smoothed away, concentrating the error at the distal end of the range (near the Bragg peak) — precisely the clinically most sensitive region. **Trading 1.47×** in speed for a **20.3% dose error is catastrophic under a 3%/3mm criterion**: this path can serve only as a "performance upper bound" reference and must not be used for any deliverable result.

Historical note (reading drift when comparing against older versions of the report): the error of this row moved with two physics corrections — **25.08%** (initial version) → **25.40%** (after the integration-grid resolution fix in `physics.fermi_eyeges_moments`) → **20.3%** (after the per-layer IDD became **fluence-normalised**, see §2). The third value is lower because fluence normalisation changed the relative weight of each energy layer in the dose, which makes the layer-weight-merged $\sigma(z)$ a better approximation; but 20.3% is still far beyond the 3% clinical tolerance, and the conclusion of this section is unchanged.

4.7 PyTorch local windowed gather (1,003.09 ms) vs Triton windowed gather + LUT (82.90 ms): 12.1× for the same algorithm

The two use exactly the same algorithm (each voxel accumulates only the beams inside a $\pm K$ neighbourhood, exploiting the compact support of the Gaussian kernel); the difference is purely implementation, and the results differ by **12.10×**:

- **The PyTorch version** (`engines.torch_gather_local`, $K=61$) performs, layer by layer, an advanced-indexing copy `w[l][idx_y,:], a T[:, idx_x, :]` expansion into an (nz, nx, Kx, ny) window copy, and then an `einsum` contraction — **every layer issues small operators on the order of $O(L \cdot nz)$** (indexing, copy, multiply, add, reduce), so launch overhead and intermediate-tensor traffic vastly exceed the actual multiply-add work: measured at only 15.6 GFLOP/s with an equivalent bandwidth of 187 GB/s, **135×** slower than the bmm exact path at the same arithmetic cost (under `operation_counts` its operation count is only 1.54× that of the separable approach).
- **The Triton version** fuses the whole chain into a single kernel: window indexing, the IDD/ σ LUTs, Gaussian kernel evaluation and weight sampling are all done on the fly inside the kernel, with only per-`kz` output, a 13×13 window $\times$ 40 layers.
- Precision is relaxed from 2.62×10^{-7} to 1.03×10^{-6} , still on the order of 10^{-6} .

But 82.90 ms is still 11.2× slower than the exact cuBLAS path. This is one of the most valuable negative conclusions in this report: on this GPU the algorithm family "independent gather per voxel" **loses to**

"**separable + dense batched GEMM**" — the former puts its parallelism on voxels and re-reads the beam table, while the latter puts its parallelism on the batch dimension and lets the hardware do regular matrix multiplication.

4.8 Triton separable convolution (fused Gaussian GEMM): 19.59 ms — 2.6× slower than cuBLAS, honestly attributed

The in-house `_pass1_z / _pass2_z` generate the Gaussian kernel on the fly inside the kernel and fuse it into `t1.dot`, so **the kernel matrices never land in VRAM**: materialising `Gx` and `Gy` the way the PyTorch path does would cost $(40,128,128,41)$ fp32 $\approx$ **102.5 MB** each, so **205 MB** of VRAM and write traffic across the two are avoided entirely. It is **2.64× slower** than the exact cuBLAS bmm path, and the attribution is specific and verifiable:

- The implementation launches pass1 and pass2 per `kz` layer, for **$2 \times 128 = 256$ kernel launches** in total;
- Each launch has a grid of only $(\lceil 128/64 \rceil, \lceil 41/32 \rceil) = 2 \times 2 = 4$ CTAs (pass2 likewise 4), giving less than 9% occupancy across **46 SMs** and leaving it dominated by launch latency and tail effects.

Honest conclusion: at this shape cuBLAS beats the hand-written Triton GEMM (cuBLAS puts all 5,120 small GEMMs of the batch into a single call, filling the GPU with several thousand CTAs at once). The value of the in-house kernels lies in **fusion (saving 205 MB of kernel matrices), controllable VRAM, and validating the CUDA optimisation strategy**, not in absolute speed; their 1.05×10^{-6} accuracy is implementation fidelity measured against the "single-sigma approximation" reference and **does not represent physical exactness**.

4.9 Triton naive gather: full-M extrapolation 654 ms

Each output voxel loops over **all M** pencil beams in registers (a row of voxels with `BLOCK_X=32` shares the same beam). The full size cannot be measured, so the scaling measurement is:

M	256	1,024	4,096
Time [ms]	3.70	10.75	40.87

Slope 9.2–9.8 μ s/beam; a linear fit extrapolated to $M = 67,240$ gives **654.19 ms** (the extrapolation assumes the spot table remains L2-resident and that cache behaviour is unchanged — an assumption that holds at $M = 4,096$ but was never verified by a full-M measurement). This kernel's arithmetic intensity is only **0.083 FLOP/B**, yet it reaches 420 GFLOP/s (measured at $M=4096$) — because the beam table it re-reads is only **0.77 MB (x/y/w) / 1.03 MB (including the int32 layer index)**, fully L2-resident. This is the core evidence for §5.

5. Roofline and Bottleneck Analysis

 Speed-up chain and roofline analysis

Figure 1 `results/figures/fig5_performance.png` (generated by `pbdose/viz.py::fig_speedup`). (a) Log-scale bar chart of median time along the speed-up chain, with the dashed line marking the 100 ms target:

red = CPU, green = PyTorch GPU, blue = Triton. (b) Roofline: arithmetic intensity [FLOP/byte] on the x-axis, measured achieved performance [GFLOP/s] on the y-axis; the black dashed line is the DRAM roofline (551.9 GB/s), the grey dotted line is the FP32 GEMM peak (14.8 TFLOP/s @4096³), the purple dash-dot line is the measured small-batch bmm ceiling (4.11 TFLOP/s @batch 5120), and the title states this section's conclusion — **the naive gather crosses the DRAM roof through cache reuse**.

5.1 Arithmetic Intensity and Roofs

Algorithm family	FLOP per voxel	Bytes per voxel	Arithmetic intensity [FLOP/B]	DRAM roof [GFLOP/s] @551.9 GB/s
Naive gather	134,480	1,613,760	0.0833	46.0
Windowed gather (theoretical)	7,442	89,304	0.0833	46.0
Separable (two passes)	108.27	549.81	0.1969	108.7

Against the roofs: FP32 GEMM peak **14.84 TFLOP/s** (4096³; also 15.46 TFLOP/s @2048³); **measured small-batch bmm ceiling 4.11 TFLOP/s** (batch 5120, 41×41×128), and only 0.63 TFLOP/s at batch 128.

5.2 Measured Achieved Performance

Implementation	Time [ms]	Achieved [GFLOP/s]	Equivalent bandwidth [GB/s]	Fraction of DRAM roof
Triton naive gather (extrapolated)	654.19	431.1	5,173	9.4× (over the roof)
Triton naive gather (measured at M=4096)	40.87	420.4	5,044	9.1× (over the roof)
Triton windowed gather + LUT	82.90	188.3	2,259	4.1× (over the roof)
PyTorch local windowed gather	1,003.09	15.6	187	0.34×
PyTorch bmm FP32 exact	7.41	30.6	156	0.28×
PyTorch bmm single-sigma	5.04	45.1	—	0.41×
Triton separable convolution	19.59	11.6	—	0.11×
PyTorch bmm CPU (reference)	395.49	0.574	—	—
NumPy separable (reference)	6,708.46	0.0338	—	—
Naive CPU (extrapolated)	1,152,984	0.245	2.94	—

5.3 Key Findings (three)

Finding 1: the naive gather crosses the DRAM roof, proving the working set is L2-resident. At an arithmetic intensity of 0.083 FLOP/B the DRAM roof is only 46.0 GFLOP/s, whereas the measurement reaches **420.4 GFLOP/s (measured at M=4096) / 431.1 GFLOP/s (full-M extrapolation)** — **more than 9× over the roof, with an equivalent bandwidth > 5 TB/s (5.04–5.17 TB/s)**. That is far beyond the measured DRAM bandwidth of 551.9 GB/s, and the only explanation is that the "24 B per tap" traffic in the roofline model is **overwhelmingly L2/register hits rather than DRAM traffic**: the coordinate/weight tables for 67,240 beams occupy only 0.77–1.03 MB and are reused over and over by every CTA. (The 286.7 GB/s of the 8 MB copy in the microbenchmark is a 58 μs-scale measurement contaminated by launch overhead and

must not be treated as an L2 roofline; this report uses it only as a point of comparison.) The 188.3 GFLOP/s of the Triton windowed gather likewise comes from L2 hits.

Finding 2: the separable path is not bandwidth-limited but limited by small-matrix GEMM efficiency and launch/dependency overhead. The exact path achieves **30.6 GFLOP/s = about 28% of its DRAM roof (108.7 GFLOP/s)**, with an equivalent bandwidth of only 156–170 GB/s; at the same time it uses only **0.7% of the measured small-batch bmm ceiling (4.11 TFLOP/s)** (within 7.41 ms the theoretical ceiling could complete 30.5 GFLOP, whereas only 0.227 GFLOP of GEMM was actually performed). Put differently: **it is neither compute-bound nor bandwidth-bound** — the bottleneck lies in the efficiency of small matrix GEMMs whose K dimension is only 41, and in the dependency chains and scheduling overhead of a great many fine-grained tensor operators. **This is the most concrete remaining optimisation opportunity in this project:** per the roofline, if this algorithm really could be pushed to bandwidth-bound, the theoretical time would be $1.153 \text{ GB} / 551.9 \text{ GB/s} \approx \mathbf{2.1 \text{ ms}}$, leaving about 3.5× of headroom.

Summary: the over-the-roof implementations (naive/windowed gather) win through cache reuse but keep re-hauling the beam table; the far-below-roof ones (bmm 30.6, Triton conv 11.6) have already reduced complexity to the minimum but cannot feed the GPU because their shapes are too small. The next step should be to **coarsen the parallelism/shapes** of the latter (§9), not to keep reducing complexity.

6. VRAM and Memory Constraints

A real OOM (CUDA out of memory): an early PyTorch implementation of the local windowed gather **expanded the entire depth direction at once**, generating a 5-D window tensor (L, nz, nx, Kx, ny) for every (layer, depth) and then applying `einsum` to it. With $L=40$, $nz=nx=ny=128$ and a window width $Kx \approx 9$:

$$40 \times 128 \times 128 \times 9 \times 128 = 7.55 \times 10^8 \text{ elements} \approx 3.0 \text{ GB (fp32)}$$

Expanding instead with the " $\pm 4\sigma$ full window" conservatively adopted in the project ($K=30 \rightarrow 61$ taps) gives $5.12 \times 10^9 \text{ elements} \approx 20.5 \text{ GB}$. The former, combined with the indexing gather copies, the fp32 dose grid (8 MB) and the workspace, triggered a real CUDA out of memory on an 11.9 GB card.

The fix (already landed): the current implementation of `engines.torch_gather_local` moves the layer loop (**layer chunking**) to the outermost level, materialising only an (nz, nx, Kx, ny) -scale intermediate T_i per layer and accumulating it into D immediately. Since L is a pure outer-product dimension of the window tensor, **peak VRAM drops by a factor of $L = 40$** ($7.55 \times 10^8 / 40 \approx 1.89 \times 10^7 \text{ elements} \approx \mathbf{75 \text{ MB}}$ scale), i.e. **roughly a 40× peak reduction**; `engines.torch_separable` also exposes a `layer_chunk` parameter for the same mechanism (`make_figures.py` verified numerical consistency with `layer_chunk=4`).

Reconciliation with the project proposal (VRAM Exhaustion / Beam Batching): this is exactly the pitfall listed in section 5 of the proposal — "if the intermediate expansion tensors of ten-thousand-scale pencil beams on a 3D grid are all kept in VRAM, VRAM will easily overflow; the countermeasures are **Beam Batching** or **On-the-fly Ray-tracing**". This project's approach is **batching along energy layers (layer chunking) + generating Gaussian kernels on the fly inside the kernel** (§4.8); both are implemented and numerically verified.

Measurement gap: in this round every row of `benchmark_full.json` has `mem_mb = 0.0` — `benchmark.gpu_mem_mb()` (wrapping `torch.cuda.max_memory_allocated()`) is implemented, but `run_benchmark.py` never writes it into the row records. The VRAM figures above are therefore **analytic estimates derived from tensor shapes**, not measured peaks.

7. Numerical Accuracy Budget

Ground truth is the NumPy float64 per-layer exact solution.

Path	Precision	Max relative error	RMS relative error	Margin vs the 3% gamma criterion
PyTorch bmm GPU per-layer exact (FP32)	float32	1.51×10^{-7}	1.33×10^{-9}	$2.0 \times 10^5 \times$ (about 5.3 orders of magnitude)
PyTorch bmm GPU TF32	tf32	1.51×10^{-7}	1.33×10^{-9}	same as FP32 (see §4.5: not actually enabled)
PyTorch local windowed gather	float32	2.62×10^{-7}	1.98×10^{-9}	$1.1 \times 10^5 \times$
Triton windowed gather + LUT	float32	1.03×10^{-6}	1.59×10^{-8}	$2.9 \times 10^4 \times$ (about 4.5 orders of magnitude)
Triton separable convolution	float32	$1.05 \times 10^{-6} \dagger$	1.41×10^{-8}	$2.9 \times 10^4 \times$
PyTorch bmm CPU per-layer exact	float64	5.37×10^{-16}	3.50×10^{-18}	reference-implementation level
PyTorch bmm single-sigma approximation	float32	2.03×10^{-1}	3.78×10^{-3}	fails ($6.8 \times$ over the criterion)

† Fidelity relative to the "single-sigma approximation" reference solution, not to the physically exact ground truth.

Conclusion: the maximum relative error of every float32 accelerated path is on the order of $10^{-6} - 10^{-7}$, which is **4.5–5.3 orders of magnitude** below the 3% gamma criterion (3×10^{-2}) — more margin than the acceptance headroom of typical TPS dose calculation. Therefore:

- float32 is sufficient precision for this problem** — the error budget is dominated by the physical model (Fermi-Eyges approximation, IDD parameterisation), not by the floating-point format;
- what pushes the error towards the criterion's red line is **algorithmic approximation** (single- σ merging, 20.3%), not the precision format; this draws the boundary — "fp32 may be traded for speed, physical approximation may not";
- if bf16 were introduced in future (8-bit mantissa, about 4×10^{-3} relative error), the margin would shrink to about $7 \times$, and end-to-end verification would have to come first (§9c).

8. Reconciliation with Project Targets

Project metric	Target	Measured	Conclusion
Single full-size dose reconstruction	< 100 ms	7.41 ms (PyTorch bmm per-layer exact FP32)	✓ 13.5× margin
Speed-up vs serial CPU baseline	the higher the better	155,544× (1,152,984.18 → 7.41 ms)	✓
Pure algorithmic speed-up (separable convolution)	—	1,242.1× ($1.410 \times 10^{11} \rightarrow 1.135 \times 10^8$ MAC)	✓ mathematical upper bound
Hardware acceleration contribution	—	$\approx$ 125.2× (155,543.7 / 1,242.1)	✓ includes BLAS/multithreading/GPU FP32
Physical exactness	per-layer exact (no σ merging)	max rel err 1.51×10^{-7}	✓
Clinical accuracy	3%/3mm gamma > 98%	numerical accuracy margin 5.3 orders of magnitude (no numerical risk; gamma analysis belongs to the accuracy report)	✓ no numerical risk
Custom GPU kernels	deliverable	three kernels — Triton windowed gather, separable convolution, naive gather (JIT→sm_120 CUBIN)	✓ but absolute speed does not beat cuBLAS (\$4.8)

Factor decomposition of the speed-up chain (the factors multiply and close exactly): $171.9\times$ (algorithm: naive→separable) $\times$ $17.0\times$ (implementation: NumPy→BLAS multithreaded) $\times$ $53.4\times$ (hardware: CPU→GPU FP32) = $155,544\times$ — taking the unrounded ratios from the JSON, the three factors give $171.8702 \times 16.9625 \times 53.3535 = 155,543.66\times$, matching the measured total speed-up digit for digit (§1.2: all ratios come from the same run). An equivalent alternative split is $1,242.1\times$ (algorithm) $\times$ $125.2\times$ (hardware + precision + implementation); the $125.2\times$ contains a $7.23\times$ "unrealised algorithmic gain", showing that about $7.2\times$ of the algorithmic benefit is still stuck in implementation efficiency (§5, Finding 2).

9. Further Optimisation Opportunities (ranked by value for effort)

(a) Eliminate the 256 kernel launches (highest priority). The Triton separable convolution issues pass1 + pass2 in a loop over `kz`, a total of $2 \times 128 = 256$ launches, while each launch has a grid of only **4 CTAs (2×2)**, giving less than 9% occupancy on 46 SMs. Fuse the two passes into a single kernel, or make `kz` the second/third `program_id` dimension so that **one launch covers all 128 depths**, or merge the submissions with **CUDA Graphs**. This is the most direct route from 19.59 ms towards cuBLAS's 7.41 ms, and it changes no numerical result.

(b) Design the top level around the measured small-matrix GEMM ceiling: σ -binning compresses L=40 into 4–8 convolutions. The hard constraint is that a small-batch bmm of $41 \times 41 \times 128$ reaches only **4.11 TFLOP/s** (batch 5120), just **27.7%** of the 14.84 TFLOP/s large-GEMM peak, while the exact path must run 40 layers. Grouping layers with similar σ into 4–8 bins (each bin sharing one kernel) would cut the number of GEMMs by 5–10×. This route shares its origin with §4.6, but **thresholds must be set strictly**: the global

single- σ merge errs by 20.3%, so binning error must be verified case by case against the 3%/3mm gamma criterion.

(c) fp32 $\rightarrow$ tf32/bf16 must first resolve "did it take effect" and "is it accurate enough". This round TF32 gave zero benefit with bit-identical errors (\$4.5), indicating that the switch had no effect; even once fixed, the small-matrix ceiling of (b) caps the gain on 227 MFLOP at under 0.1 ms. bf16's roughly 4×10^{-3} relative error is still within 3%, but leaves a margin of only $\sim 7\times$.

(d) Use `cp.async` / TMA for explicit pipelining on sm_120. Triton's `tl.dot` loops are already software-pipelined automatically, so the most likely explicit gain is in the **windowed gather** ($13 \times 13 \times 40 = 6,760$ tap accesses per voxel, with a large fraction hitting L2). **Precondition:** this machine lacks Nsight Compute, so counters cannot be used to localise stall types; add counters before modifying the kernel.

(e) Replace the two bmms with banded/sparse kernels. `Gx` and `Gy` are Gaussian kernels, strictly 0 beyond $\pm 4\sigma$; for a representative $\sigma \approx 1.6$ cm only **about 13** of the 41 points contribute (the worst case is still 33 out of 41). **Measured evidence:** the Triton windowed gather with a tight ± 3 cm window (`kx=ky=6`, 13 taps) achieves a maximum relative error of only 1.03×10^{-6} against the exact float64 reference, showing that wherever large σ occurs the weights are negligible. Compressing the contraction dimension from 41 down to ~ 13 would cut the FLOP of these two bmms to about 1/3 — orthogonal to (a) and additive with it.

10. Reproduction Commands

On a machine with an NVIDIA GPU (sm_120) + PyTorch 2.11.0+cu128 + triton-windows, run the following from the repository root:

```
# 1) Full-size speed-up chain benchmark (N=15 repetitions, median + min), produces results/benchmark_full
python scripts/run_benchmark.py --repeat 15 --tag full

# 2) Generate all scientific figures (depends on benchmark_full.json from the previous step), produces re
python scripts/make_figures.py

# 3) GPU microbenchmarks (DRAM bandwidth / L2 copy / FP32 GEMM peak / small-batch bmm), produces results/
python tools/gpu_microbench.py
```

- Step 1 is a prerequisite for step 2 (`make_figures.py` reads `results/benchmark_full.json` to generate Figure 1, falling back to `benchmark_quick.json` if it is missing); `--quick` skips the scaling measurement of the Triton naive gather and the full-M extrapolation row.
 - No CUDA Toolkit is required: the kernels are compiled to sm_120 CUBIN by the `ptxas` shipped with Triton. The reference dose fields (`dose_reference_f64.npy`, `dose_torch_gpu_f32.npy`, `dose_exact_f32_gpu.npy`) are written out by steps 1 and 2.
-

Appendix: Measurement Limitations (stated honestly)

Extrapolated values in two places (the naive CPU baseline and the Triton naive gather at full M); **peak VRAM not measured** (`mem_mb` is 0.0 for every row); **no hardware counters** (§5's attribution is inference); **the two op-count conventions disagree** (§2); **the accuracy reference for the Triton separable convolution is an approximate solution**. All of these points are flagged at the corresponding places in the body of the report.